Datenbank-Spektrum (2024) 24:197–207
https://doi.org/10.1007/s13222-024-00488-z

SCHWERPUNKTBEITRAG

Imputation Strategies in Time Series Based on Language Models

Michel Jacobsen[1] · **Marina Tropmann-Frick**[1]

Received: 30 May 2024 / Accepted: 16 September 2024 / Published online: 14 October 2024
© The Author(s) 2024

Abstract
Incomplete time series present a significant challenge for downstream analysis. In the field of time series, Large Language Models are already being used for prediction, classification, and, in rare cases, imputation. This study thoroughly examines the imputation of time series using Large Language Models. Within a defined experimental setup, current state-of-the-art time series imputation methods are compared with the performance of Large Language Models. Parameter-efficient fine-tuning methods are applied to adapt the Large Language Models to the imputation task. The results indicate that the models are suitable for time series imputation. The performance of these models depends on the number of parameters and the type of pre-training. Small specialized models, such as BERT, compete with models like Llama2 and outperform them on selected datasets. Furthermore, it becomes clear that the attention and feedforward network components of Large Language Models are particularly well-suited for adaptation to imputation, and parameter-efficient methods are also performance-enhancing.

Keywords Imputation · Missing Values · Time Series · Language Models · Fine-Tuning

1 Introduction

The imputation of incomplete time series is a crucial task for downstream analysis tasks and has a direct impact on the performance of models trained on the data. Traditional methods lead to biases in the data [1]. Methods based on deep learning and Transformer architectures have made significant advancements in time series imputation, as they can account for complex and temporal dependencies in the data. The Transformer architecture has also enabled remarkable progress in the field of Natural Language Processing (NLP). Large Language Models (LLM) are very powerful due to their model size and training methods [24]. In this context, there are efforts to transfer the advancements of these models from the NLP to the time series domain. The transfer of LLMs to time series is possible due to the nature of the models, which predict the next token of a sequence [7]. Additionally, the models are capable of processing sequences of length N. The results of other studies show that LLMs can achieve leading results compared to other state-of-the-art imputation methods. However, most of the work uses LLMs only for forecasting or classification of time series [2, 3, 7]. Time series imputation with LLMs is considered in [26] and shows remarkable results.

These results will be further explored in this paper, and specific aspects of time series imputation with LLMs will be examined. Within a defined setup, different LLMs are compared to determine which factors enhance leading performance. Furthermore, the individual model components of a LLM will be optimized, and their impact on the imputation capability will be analyzed. Another question focuses on the choice of fine-tuning method. NLP fine-tuning methods are applied to imputation and tested for their applicability in this context.

The paper is structured as follows. Following the introduction, relevant contents of research are presented. Subsequently, the framework that serves as the basis for the experiments and enables the exchange of different LLMs is presented (see Sect. 3). Sect. 4 describes the setup of the experiments and the results. Sect. 5 discusses these results in relation to current research. The paper concludes with Sect. 6.

✉ Michel Jacobsen
michel.jacobsen@haw-hamburg.de

Marina Tropmann-Frick
marina.tropmann-frick@haw-hamburg.de

[1] Department of Computer Science, Hamburg University of Applied Sciences (HAW Hamburg), Berliner Tor 5, 20099 Hamburg, Hamburg, Germany

2 Related Work

2.1 Time Series Imputation

Time series data, sequences of observations collected over time, are widespread in various fields such as finance, economics and medicine. The analysis of these time series relies on the data being complete [11]. However, missing data frequently occur due to sensor malfunctions or input errors [12]. Filling in the missing values is known as imputation and ensures the integrity and usability of the data [11].

Traditional and simple techniques, such as mean imputation or the removal of these datasets, can introduce biases and lead to distortions. This directly impacts the quality of subsequent analysis methods [12]. Therefore, more advanced methods have been developed that better capture temporal complexities. Machine learning (ML)-based imputation methods surpass traditional methods by learning complex patterns and relationships within the data. These methods include, for example, k-nearest neighbors (kNN) and support vector machines (SVMs) and use supervised learning to approximate missing values based on temporal information. Despite the advances of these methods, capturing long-term dependencies in time series remains challenging [15].

The emergence of deep learning has further revolutionized the field of time series imputation. Deep learning models, particularly Recurrent Neural Networks (RNNs) and their variants like Long Short-Term Memory (LSTM) networks, have shown remarkable capabilities in handling sequential data [12]. In recent years, attention-based models, such as Transformers, have set new benchmarks in various sequence modeling tasks. Their ability to model relationships between all elements in a sequence simultaneously allows them to capture dependencies more effectively than traditional RNNs [20]. This characteristic makes Transformer models highly suitable for imputation tasks, where understanding the context of missing values is crucial. A Transformer-based approach to time series imputation is SAITS [6]. The model uses the Transformer as its foundation and employs different loss functions to learn the task of imputation in detail. As one of the state-of-the-art models, SAITS is specialized in the imputation of time series [6].

2.2 LLMs and Imputation Approaches with LLMs

LLMs are models designed to represent concepts in natural language and are developed within the field of NLP [24]. Typically, these models are trained to predict the next token in a sequence. Due to their complexity and size, the most advanced models have made remarkable progress in recent years [24].

The fundamental architecture of these models is the Transformer architecture [24]. It is this architecture, with the help of the attention mechanism, that has enabled the capture of long sequences and the incorporated temporal dependencies [20]. The Transformer consists of various components. The components of the architecture include an attention block, an add & normalization layer (LN), and a feedforward network (FFN) [20]. These enable the processing of sequences in an encoder-decoder structure. To develop LLMs, modifications are made to the original Transformer architecture, such as the use of modified attention mechanisms or different training methods. LLMs can generally be trained in two ways. Language modeling is often used in decoder-only architectures and involves predicting the next token in a sequence based on the preceding tokens. In contrast, with denoising autoencoding, the model has access to the entire sequence, but parts of the sequence are altered or masked. The model learns to restore the original sequence [24].

The fine-tuning of an LLM and the adaptation to other tasks require the use of fine-tuning methods. With respect to LLMs, the application of parameter-efficient methods is necessary because fully fine-tuning large models is very computationally intensive. Low-rank adaptation (LoRA) is a popular method that replaces the update matrix in the fine-tuning process with a decomposition, thereby reducing the complexity of the necessary matrix [8]. In addition to LoRA, there are several other methods that are applied in the context of the third experiment.

There are two main strategies for applying LLMs to time series imputation: multimodal repurposing and API-based prompting. Multimodal repurposing involves fine-tuning LLMs specifically for the imputation task, sometimes adding new components to the models to better handle time series data. This method can significantly enhance the models performance by aligning their pre-training objectives with the target task. On the other hand, API-based prompting involves combining time series data with natural language inputs to guide the model in filling in the gaps. This approach is less resource-intensive as it leverages pre-existing models with minimal modifications [24].

3 Methodology

The experimental approach necessitates a modular architecture, enabling the substitution of the central component, the LLM, to ensure comparable results. To meet these requirements, the architecture of GPT4TS [26] is adapted with necessary modifications. At the core of the framework is an LLM, with various models employed for comparison as detailed in Sect. 4. The selection of language models was based on three criteria: adherence to the standard

transformer architecture for detailed analysis of component fine-tuning, open-source availability for consistent parameter comparisons, and manageable computational resource requirements given the significant demands of very large models.

The LLM's foundation on transformer architecture components allows for customization of individual model parts. In the initial experiment, the attention matrices and the feed-forward network (FFN) are fixed and non-trainable. Customization is achieved through components external to the LLM, such as the input embedding, positional embeddings, and output linear layer, as well as using LoRA within the transformer blocks and the add & layer norm component. This integration of parameter-efficient fine-tuning (PEFT) methods marks a deviation from GPT4TS.

Zhou et al.'s architecture includes instance normalization and a trainable input embedding to transform the time series data into a format suitable for LLM processing [26]. Positional embeddings are also adjusted during fine-tuning. Upon completion of the time series processing by the LLM, an output layer maps the data back into a format suitable for model quality validation (see Fig. 1). For detailed information on the individual components, refer to Zhou et al. [26].

The datasets used in this work include Electricity Transformer Temperature (ETT), Electricity Consuming Load (ECL), and a Weather dataset, each providing real-world data for model fine-tuning and evaluation [25]. The selection of these datasets is based on their use in previous studies [21, 26], ensuring consistency and comparability. The ETT data set is divided into four subsets (ETTh1, ETTh2, ETTm1, ETTm2), so that the comparisons of the language models take place on a total of six data sets. The subsets only differ in the measurement frequency, which is hourly (h) and minutely (m). In addition, the data includes two geographical locations (1 and 2). Due to the nature of the data, there are consistent trends with seasonal components (e.g., weather patterns, electricity consumption). Additionally, the data contains very few outliers, so no special handling is required.

A significant reason for selecting these datasets is their completeness, which allows for manipulation to simulate missingness and subsequent imputation. The generation of missing data is randomized and performed at varying proportions (12.5%, 25%, 37.5%, 50%) depending on the experiment. This procedure aligns with the methodologies used in SAITS [6], TimesNet [21], and GP4TS [26], ensuring consistency in experimental design.

In the experiments, the time series of the datasets are segmented into sequences of 96 time steps. Sequences are masked randomly, based on the predefined masking rates. While the masking rate determines the extent of missing values, the pattern of missing data remains entirely stochastic, with no control over whether individual points or continuous segments are removed. Following the masking of data points, the incomplete time series are input into the model for imputation. Upon completion, the error metric is computed by comparing the imputed values to the original time series, serving as the basis for optimization during the training process. Once the training phase is concluded, the optimized models are evaluated on a separate evaluation dataset. In this evaluation phase, data points are again randomly masked, and the corresponding performance metrics are collected for analysis. Although the data in this setting is multivariate, the methodology is adaptable to any type of time series, provided the data is appropriately preprocessed for the language models. All other studies do not provide detailed descriptions of the masking process, stating only that it is performed randomly.

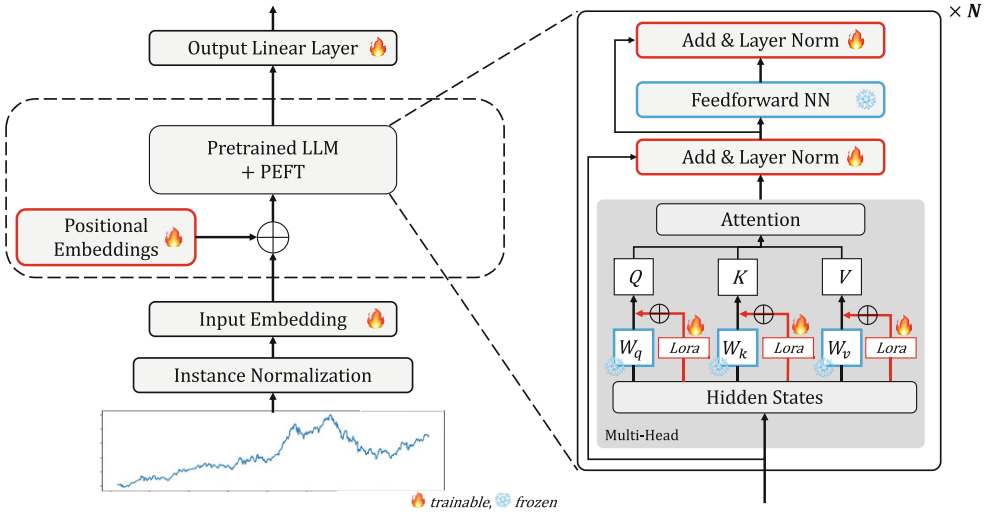

Fig. 1 Adapted Architecture from Zhou et al. [26]. Unlike GPT4TS, it includes LoRA components that allow fine-tuning of the attention layer in the pretrained LLM

4 Experiments

This section presents the experiments and is divided into two parts. The first part describes the experimental setup, such as the datasets and the LLMs used. The second part presents the results of the experiments.

4.1 Experimental setup

Datasets and baselines. The experimental setup comprises various datasets and baseline models to evaluate the performance of different LLMs for imputation on time series data. For comparative analysis of specialised models with LLMs the state-of-the-art imputation models SAITS [6] and TimesNet [21] were used. Also a vanilla transformer is employed.

LLMs. GPT-2 is a Generative Pretrained Transformer (GPT) based on the Transformer architecture, using only the decoder part. The base model has 12 layers, 12 attention heads, and a model dimension of 768, comprising around 117 million parameters. Trained on approximately 40 GB of data (8 million documents), GPT-2 is unidirectional, meaning it can only access previous tokens. It was pre-trained using the language modeling approach and then fine-tuned for specific NLP tasks [17].

BERT is an encoder-only model known as **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike GPT-2, BERT captures bidirectional context, enhancing language understanding. It has 12 layers, 12 attention heads, and a model dimension of 768, with about 110 million parameters. BERT's pre-training involves Masked Language Modeling, where random tokens in a sequence are masked and the model is trained to reconstruct them, and Next Sentence Prediction. Following pre-training, BERT is fine-tuned for various NLP tasks [5].

T5 builds on the classic Transformer architecture, using both an encoder and a decoder with minor modifications. Each has 12 layers, and the model has 220 million parameters. Similar to BERT, T5 is pre-trained by manipulating input sequences and masking tokens. It employs a Multi-Task Pre-Training approach, allowing it to learn different tasks during training [18].

BART is trained using the denoising autoencoding approach. It uses an encoder-decoder structure, each with six layers and a model dimension of 768. Pre-training involves altering the input sequence through token deletion, masking, or rotation, then reconstructing the original sequence. BART is then fine-tuned for specific NLP tasks such as classification and text generation. The base model has about 140 million parameters [13].

The next section presents two large language models (LLMs) that significantly differ from the previously described models in terms of the number of parameters. Another distinguishing feature of these models is the scale of the training data used for pre-training. Unlike earlier datasets, which contained up to 130 billion tokens, these models were optimized with training data on the order of trillions of tokens.

Llama2, released by Meta, is a publicly accessible LLM that competes with proprietary models in certain benchmarks. It comes in three sizes (7, 13, and 70 billion parameters), significantly exceeding the parameter counts of previous models. As a pure decoder model with 32 attention heads and 32 layers, Llama2 has a model dimension of 4096. It was trained on a dataset of two trillion tokens. The model's architecture includes optimizations for enhanced performance, focusing on language modeling to predict the next word in a sequence. Further fine-tuning methods include instruction tuning and reinforcement learning with human feedback (RLHF). For the experiments, the smallest Llama2 model is used due to resource constraints [19].

Phi-2 is a large language model with two billion parameters, similar in architecture to Llama2. It is a decoder-only model with 24 layers and 32 attention heads, utilizing Flash Attention instead of the traditional attention mechanism. The pre-training task is language modeling, focusing on predicting the next word in a sentence. Notably, Phi-2 has not undergone fine-tuning. Despite this, Microsoft has demonstrated that Phi-2 outperforms previous models in certain benchmarks due to its use of high-quality training data. This shows that high-quality data can compensate for smaller model size [10].

The experimental setup involves adapting the models for imputation within the constructed framework (see Fig. 1). Positional embeddings and the Add & LN components of the transformer blocks are adjusted, along with attention blocks using the parameter-efficient LoRA method. Models are fine-tuned using Mean Squared Error (MSE) and additionally validated with Mean Absolute Error (MAE). Following Zhou et al. [26], only a quarter of the original model layers are used for optimization: GPT-2, BERT, and T5 have 3 layers each; Llama2 and Phi-2 have 8 layers each. Note that Phi-2, initially described with 24 layers, is provided by Microsoft on Huggingface as a 32-layer model. BART, as an encoder-decoder model, has 12 layers, split equally between the encoder and decoder, leaving two transformer blocks in each component.

4.2 Results

The first experiment compares the imputation performance of pretrained language models among themselves and against the baseline models. The models described in the previous section are applied within the framework from Sect. 3 and optimized on the datasets.

Table 1 Count of trainable parameters of LLMs

Model	Modelsize	Trainable parameters	%
GPT-2	60,759,552	906,240	1.49
BERT	45,838,848	551,424	1.20
T5	74,676,416	456,128	0.61
BART	73,556,736	1,883,136	2.56
Llama2	1,752,240,128	2,162,688	0.12
Phi-2	761,758,720	1,351,680	0.18

Due to the use of only 25% of the original model sizes of the LLMs, the number of model parameters is reduced. Additionally, the application of LoRA as a parameter-efficient fine-tuning method results in only the added adapters within the attention blocks being trainable. This leads to different numbers of adjustable parameters for the models, which are summarized in Table 1. The table shows that up to 50% of the original parameters remain (e.g., GPT-2, originally 120M parameters). The adaptive parameters are not frozen during the optimization process and are trained for the imputation task. For example, in the Llama2 model, only 0.12% of the parameters are trainable. Despite this, approximately two million adaptive parameters result overall.

The proportion of parameters depends on the model architecture and the internal model dimensions.

4.2.1 Imputation performance of LLMs

In this section, the results of the performance comparison are presented. Table 2 summarizes the findings. The overview of the results shows that no single model dominates across all datasets. It is evident that the specialized models SAITS and TimesNet deliver consistently good results. SAITS outperforms other models on two datasets (ETTh1 & ETTm1). Other strong performers include BERT and TimesNet based on MSE, and Phi-2 based on MAE. The fluctuating distribution of the lowest errors also indicates that none of the LLMs consistently achieves the second-best score in the experiments. Instead, models such as BERT, Llama2, Phi-2, and T5 take turns in this position. However, it is clear that the models GPT-2 and BART rarely achieve the best results compared to other models. It is also apparent that the classic Transformer model does not perform at the same level as the other models. Although it delivers comparable performance in some experiments, it never manages to position itself as a leader. The best performances are achieved by SAITS and the LLMs Phi-2, BERT, and T5 (see Table 2).

As the results presented are not unambiguous, the performance of the LLMs in particular is compared below. This should enable a better understanding of the subset of LLMs and provide a better impression of the performance of these models. The analysis of the experimental results indicates that the lowest error rates are recorded both for the large language models Llama2 and Phi-2, as well as for the significantly smaller models BERT and T5. A trend emerges showing that either the larger or the smaller models dominate. Specifically, high performance of a large language model like Llama2 typically correlates with similarly high performance from Phi-2 (see Table 2 for datasets ETTh1, ETTm1, and Electricity). In contrast, for the datasets ETTh2 and Weather, BERT and T5 primarily achieve the lowest error values. However, it should be mentioned here that Llama2 and Phi-2 play a leading role in some error rates.

A MANOVA was conducted to examine the effects of the model, dataset and mask ratio on the outcomes. The results revealed a significant multivariate effect, e.g. Wilks' Lambda = 0.4804, $F(16,1260) = 34.87$, $p < 0.001$ for model and 0.0699, $F(10,1260) = 350.4867$, $p < 0.001$ for the dataset. Post-hoc analysis indicated a significant effect of the model and dataset on the MAE and MSE. The effect of the mask ratio is less significant. The results confirm the previous statements that the lowest error values depend on the model and the dataset.

4.2.2 Fine-tuning different model components

The fine-tuning of the models from the previous section is carried out using a standardized procedure in which all models are trained equally. To investigate the second aspect of this study, a different experimental approach is introduced, focusing specifically on fine-tuning various components of the transformer architecture. The emphasis is on targeted optimization of different model layers. Based on the transformer architecture, the structure of the LLMs allows for selective optimization of individual or multiple layers. In particular, the attention layer, the FFN, and the Add & LN component of the transformer block are examined both individually and in combination. The GPT-2 model [17], which was used in the first experiment, serves as the basis for this investigation. Additionally, the architecture of GPT-2 aligns with the basic structure of the large leading language models (here, Llama2, Phi-2), making the results transferable to these models. The GPT-2 model has approximately 120 million parameters. By adjusting and fine-tuning three of the twelve possible layers, the number of parameters is reduced to about 60 million. During the experiments, the input embeddings of the model (word token embedding, word positional embedding) are always optimized. This results in a baseline number of adjustable parameters. Additionally, in each run, a specific component or a combination of components is fine-tuned. The results of the component training are summarized in Table 3 in an aggregated form per dataset. The summary shows that fine-tuning the model and adjusting it for imputation does

Table 2 Full imputation results for all models on every dataset and different mask ratios

Models		GPT-2		BERT		Saits		TimesNet		Transformer		Llama2		Phi-2		T5		BART	
Mask Ratio		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	12.5%	0.052	0.154	0.047	0.146	**0.028**	**0.112**	0.068	0.176	0.059	0.172	*0.041*	*0.138*	0.042	0.139	0.051	0.155	0.053	0.155
	25%	0.074	0.182	0.064	0.170	**0.039**	**0.132**	0.085	0.195	0.082	0.204	*0.052*	*0.153*	0.055	0.159	0.069	0.178	0.067	0.175
	37.5%	0.098	0.207	0.084	0.194	**0.047**	**0.143**	0.100	0.212	0.108	0.232	*0.069*	*0.175*	0.070	0.180	0.084	0.196	0.092	0.205
	50%	0.123	0.232	0.107	0.216	**0.064**	**0.168**	0.121	0.231	0.135	0.261	*0.093*	*0.204*	0.096	0.208	0.105	0.218	0.115	0.225
ETTh2	12.5%	0.041	0.131	**0.040**	**0.128**	0.063	0.171	0.041	0.136	0.168	0.299	0.041	0.131	0.041	*0.130*	*0.040*	0.131	0.041	0.132
	25%	0.050	0.146	**0.046**	*0.141*	0.061	0.172	0.048	0.147	0.228	0.348	0.048	0.143	*0.046*	**0.138**	0.047	0.144	0.047	0.143
	37.5%	0.057	0.158	*0.052*	0.152	0.091	0.211	0.054	0.156	0.287	0.391	0.053	0.152	0.052	**0.149**	**0.052**	*0.151*	0.053	0.153
	50%	0.067	0.172	0.061	0.165	0.108	0.232	0.062	0.167	0.311	0.410	0.061	0.163	0.061	*0.162*	**0.058**	**0.161**	*0.060*	0.164
ETTm1	12.5%	0.024	0.105	0.020	0.093	**0.015**	**0.082**	0.020	0.093	0.021	0.101	*0.019*	0.091	0.019	*0.090*	0.024	0.105	0.022	0.100
	25%	0.033	0.121	0.024	0.103	**0.016**	**0.084**	0.024	0.102	0.028	0.116	*0.023*	0.099	0.023	*0.098*	0.030	0.117	0.027	0.111
	37.5%	0.041	0.136	0.030	0.115	**0.018**	**0.088**	0.030	0.114	0.035	0.130	*0.029*	*0.109*	0.029	0.109	0.036	0.126	0.033	0.122
	50%	0.053	0.153	0.036	0.128	**0.029**	**0.114**	0.037	0.126	0.044	0.146	*0.036*	*0.121*	0.038	0.126	0.042	0.138	0.041	0.135
ETTm2	12.5%	0.021	0.087	**0.019**	**0.080**	0.023	0.097	*0.019*	*0.083*	0.150	0.284	0.021	0.087	0.023	0.090	0.020	0.087	0.020	0.086
	25%	0.025	0.097	**0.021**	**0.085**	0.030	0.117	*0.021*	*0.087*	0.157	0.282	0.024	0.092	0.023	0.091	0.023	0.092	0.023	0.093
	37.5%	0.028	0.104	*0.023*	**0.093**	0.046	0.147	**0.023**	*0.093*	0.226	0.345	0.025	0.095	0.027	0.100	0.025	0.098	0.025	0.097
	50%	0.031	0.111	*0.026*	*0.100*	0.056	0.166	**0.026**	**0.099**	0.222	0.347	0.029	0.105	0.030	0.107	0.028	0.105	0.028	0.105
Electricity	12.5%	0.072	0.186	0.071	0.186	0.146	0.272	0.089	0.204	0.147	0.276	**0.065**	**0.177**	*0.065*	*0.177*	0.080	0.198	0.078	0.193
	25%	0.080	0.196	0.079	0.196	0.161	0.281	0.097	0.211	0.159	0.283	*0.077*	*0.192*	**0.075**	**0.190**	0.087	0.205	0.084	0.200
	37.5%	0.089	0.207	0.087	0.205	0.172	0.288	0.098	0.215	0.165	0.287	*0.085*	*0.203*	**0.085**	**0.202**	0.093	0.213	0.091	0.209
	50%	0.096	0.216	0.096	0.216	0.181	0.295	0.104	0.222	0.179	0.297	**0.094**	*0.214*	*0.094*	**0.214**	0.100	0.222	0.099	0.218
Weather	12.5%	**0.026**	*0.048*	0.027	0.050	0.028	0.064	*0.026*	**0.048**	0.032	0.087	0.027	0.048	0.080	0.124	0.026	0.050	0.030	0.058
	25%	0.030	0.055	0.030	0.055	0.031	0.069	**0.029**	**0.053**	0.034	0.084	0.077	0.120	0.031	0.056	*0.029*	*0.054*	0.034	0.065
	37.5%	0.033	0.060	0.032	0.059	0.050	0.113	**0.032**	**0.058**	0.037	0.087	0.035	0.064	0.087	0.137	*0.032*	*0.059*	0.035	0.065
	50%	0.037	*0.067*	0.077	0.118	0.039	0.086	*0.036*	0.064	0.040	0.091	0.039	0.070	0.084	0.127	**0.035**	**0.063**	0.043	0.080
Count 1st		1	0	*4*	4	**8**	**8**	*4*	4	0	0	2	1	2	*5*	3	2	0	0
Count 2nd		0	2	3	2	0	0	*4*	3	0	0	**10**	**9**	3	*5*	3	3	1	0

Table 3 Aggregated results per dataset for the fine-tuning of different components and their respective combinations

Dataset	Attention (1)		Add & LN (2)		FFN (3)		(1 & 3)		(2 & 3)		(1 & 2)		(1 & 2 & 3)	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	0.0767	0.1842	0.1005	0.2064	0.0734	0.1795	**0.0688**	**0.1724**	0.0698	0.1743	0.0753	0.1815	*0.0691*	*0.1730*
ETTh2	0.0513	0.1481	0.0563	0.1564	0.0510	0.1453	**0.0489**	**0.1415**	0.0499	0.1435	0.0506	0.1466	*0.0490*	*0.1418*
ETTm1	0.0316	0.1163	0.0454	0.1396	0.0293	0.1095	**0.0274**	**0.1058**	0.0287	0.1083	0.0311	0.1156	*0.0281*	*0.1072*
ETTm2	0.0248	0.0955	0.0279	0.1036	0.0228	0.0884	**0.0214**	**0.0854**	0.0224	0.0872	0.0241	0.0940	*0.0214*	*0.0854*
Electricity	*0.0844*	*0.2009*	0.0854	0.2033	0.0896	0.2056	0.0907	0.2073	0.0901	0.2064	**0.0843**	**0.2006**	0.0911	0.2075
Weather	0.0308	0.0555	0.0321	0.0581	0.0305	0.0545	0.0304	**0.0538**	**0.0302**	*0.0542*	0.0306	0.0554	*0.0303*	0.0540

*Legend: **best value**, *second best value*

not significantly improve by optimizing a single component alone. Optimizing the attention (1) or FFN (3) layers generally leads to better imputation performance compared to fine-tuning using the Add & LN component. This behavior is observed in five of the six datasets. On average, the combination of the attention and FFN components performs best (see Table 3, column 1 & 3). This performance is particularly evident on the ETT datasets, where fine-tuning these components yields leading results.

Regarding the other datasets (electricity, weather), the results are less consistent. For the imputation of the electricity dataset, the combination of attention and Add & LN (see column (2 & 3)) achieves the best result. For the weather dataset, the differences in the restoration of masked time series are minimal, and all variations are within a comparable range. Although the combination of Add & LN and FFN shows a slightly lower error rate, the results are homogeneous.

For the fine-tuning of the model components, the results of a MANOVA test also confirm the significant effect of the different components (Wilks' Lambda = 0.5471, $F(12,976) = 28.6240$, $p < 0.001$). However, the test also shows that the dependent variables (MAE, MSE) are also strongly influenced by the data set. Therefore, varying the fine-tuning of the different model components has a significant influence, although the results are more strongly driven by the datasets (Wilks' Lambda = 0.0019, $F(10,976) = 2147.2374$, $p < 0.001$).

4.2.3 Comparison of specific fine-tuning methods

In the previously presented experiments investigating the general performance of LLMs (see Sect. 4.2.1), LoRA was used as the fine-tuning method. The following section examines the impact of different PEFT methods on the imputation performance of the model. To explore this aspect, the experimental setup for the first experiment is followed, and the PEFT method is changed within the modular framework. For consistency, the GPT-2 model is again used as the base model for the necessary adjustments. The specific optimization methods examined are LoRA [8], AdaLora [22], (IA^3) [14], and the quantized version of LoRA, QLoRA [4]. Although QLoRA is not a classic PEFT method but a memory-efficient optimization method, it is included due to its widespread use in fine-tuning LLMs. The methods were applied with uniform parameters so that a standardized procedure is available and the results are comparable in every case.

Following the description of the general experimental setup, the number of adaptive parameters for each method is considered. Table 4 shows the total number of optimizable parameters and the additional parameters introduced by each method.

Table 4 PEFT tuning – adaptive parameters for each method

PEFT method	Additional parameters	Total
LoRA	110,592	906,240
AdaLora	165,960	961,608
(IA^3)	9,216	795,648
Qlora	110,592	906,240

When applying LoRA and QLoRA, approximately 110,000 parameters are added. The AdaLora method, which follows a more flexible approach, results in a larger number of added parameters (approximately 166,000). The (IA^3) method significantly deviates from the other methods and adds only 9,216 parameters to the model.

At an aggregated level per dataset, Table 5 summarizes the results. In detail, the results indicate that QLoRA and LoRA methods lead to very similar error rates, showing leading results particularly on the ETT and electricity datasets. Only on the weather dataset is a deviation observed, where the AdaLora method yields better results, although the LoRA method also performs very well. In summary, the LoRA and QLoRA methods show very consistent results. When switching datasets and thus domains, the results become more heterogeneous, and the AdaLora method also shows comparable performance (see Table 5, electricity & weather datasets). (IA^3) does not achieve leading error rates on any of the datasets. It only outperforms QLoRA on the weather dataset. Otherwise, the imputation performance decreases with the use of the (IA^3) method.

All independent variables (data set, mask ratio, PEFT method) have a significant influence on the error metrics. This is validated by a MANOVA test. This test confirms the previous statements that the performance of the model differs depending on the data set and that the PEFT method also demonstrates a clear dependency of the error metrics (Wilks' Lambda: 0.08472, $F(6,554) = 7,9809$, $p < 0.001$ for the PEFT method, 0.0009, $F(10,554) = 1766.1725$, $p < 0.001$ for the dataset). The presentation of the experimental results is followed by a discussion of the results of the individual experiments.

5 Discussion

LLM Imputation. Pretrained language models perform comparably to current state-of-the-art methods in time series imputation, often surpassing them on certain datasets. The selected LLMs in this study provide a solid foundation for imputation. Model size, measured by the number of parameters, influences the imputation of time series, as evidenced by the results. Large language models like Llama2 and Phi-2 achieve leading results on half of the datasets, while smaller models generally show limited performance, indicating that model size is crucial for performance on these datasets.

The type of pre-training is also critical for imputation performance. Models like BERT and T5 trained on approaches that corrupt input sequences (e.g., masked sequences for BERT), show that pre-training methods resembling denoising autoencoders enhance imputation capabilities. Smaller models trained this way can outperform larger models on two of the six datasets, confirming the similarity between time series imputation and denoising autoencoding for LLMs. However, BART, trained with diverse pre-training tasks like token deletion and rotation, does not consistently demonstrate this advantage, suggesting this approach may not be suitable for imputation.

The experiments demonstrate that the methodological framework (see Sect. 3) effectively enables the imputation of time series using LLMs. Both the size of the model and the type of pre-training emerge as critical determinants of performance. Notably, specialized models based on the denoising autoencoding approach prove to be as effective for time series imputation as the larger LLMs. These specialized models achieve performance comparable to that of larger language models while offering greater resource efficiency due to their smaller size. This specialization is further substantiated by the performance of the SAITS and TimesNet models, which were specifically designed for imputation and, in certain cases, deliver similar levels of performance.

Component tuning. Related work does not thoroughly examine which LLM components are best suited for fine-tun-

Table 5 PEFT tuning – aggregated results

	LoRA		AdaLora		(IA^3)		Qlora	
Dataset	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	**0.0867**	**0.1938**	0.0915	0.1973	0.1005	0.2067	*0.0885*	*0.1942*
ETTh2	**0.0538**	**0.1519**	0.0542	0.1528	0.0564	0.1569	*0.0538*	*0.1522*
ETTm1	*0.0380*	*0.1285*	0.0414	0.1339	0.0452	0.1394	**0.0376**	**0.1281**
ETTm2	*0.0264*	*0.1000*	0.0270	0.1014	0.0280	0.1039	**0.0263**	**0.0996**
Electricity	**0.0841**	**0.2013**	0.0842	0.2017	0.0856	0.2036	*0.0842*	*0.2015*
Weather	**0.0315**	*0.0573*	*0.0316*	**0.0571**	0.0318	0.0577	0.0320	0.0579

*Legend: **best value**, *second best value*

ing on time series imputation. The GPT4TS framework [26] assumes that most of the knowledge of a language model is stored in the attention layers and does not adjust this layer during fine-tuning. This study has shown that integrating PEFT methods in the attention layer can yield good results. Additionally, results from Sect. 4.2.2 indicate that adjusting the attention layer in combination with the FFN yields the highest performance, extending the experiment setup from Zhou et al. [26].

The results on the ETT datasets show that fine-tuning the Add & LN component has only a minor impact on imputation performance. However, this behavior is not observed on the other datasets (electricity, weather), where the influence of this component is inconclusive. Other studies on fine-tuning LLMs for advanced NLP tasks suggest that exclusively adjusting the Add & LN component can be effective [23], but this was not observed in the experiments of this study.

Overall, the results regarding the second aspect of this paper suggest that both the attention layer and FFN should generally be adjusted. This implies that a substantial portion of the model needs to be optimized, essentially aiming for a complete fine-tuning of the model. Contrary to the goal of adapting an LLM to a new domain with minimal parameter adjustments, the observed results suggest that extensive fine-tuning of the entire model yields the best results. This significantly increases the resource requirements for this process. These observations align with expectations from other domains and applications and can be confirmed for optimizing LLMs for imputation.

PEFT methods. To investigate the aspect of PEFT methods, various optimization methods were applied, and the results lead to the following conclusions. The superior performance of models using the LoRA method confirms its widespread use for fine-tuning LLMs. This dominance is observed across almost all datasets, resulting in very consistent outcomes. The lower performance of the (IA^3) method can be attributed to the small number of added parameters, as (IA^3) only integrates individual scaling vectors into the components of the LLMs. In contrast, other methods that add more optimizable parameters show better performance. These results were expected and are supported by other studies [9, 16], which also confirm the leading role of LoRA. Therefore, it is evident that PEFT methods are suitable for the imputation task. The used framework preprocesses the time series data before passing them to the language models, resulting in no difference compared to fine-tuning for subsequent NLP tasks.

In summary, the analysis of the performance comparison of various PEFT methods reveals that LoRA shows the best performance in the considered imputation setup. This observation is in line with expectations, considering that LoRA plays an important role in the optimization of LLMs. However, depending on the need for resource-efficient optimization, other approaches such as QLoRA or (IA^3) also present suitable alternatives.

Limitations and Future Work. Following the discussion of individual results, the limitations and constraints of this study are described. One limitation is the inability to consider the largest publicly available LLMs due to resource constraints. Models with billions of parameters can currently only be optimized on highly powerful GPUs. The resource-intensive setting of this study also necessitated the use of fine-tuning methods. Some selected models required PEFT methods as classic fine-tuning of the attention blocks could lead to bottlenecks. Consequently, models like Llama2 could not be considered for the second research question. These limitations were mitigated by including the largest possible models and comparing language models with different pre-training concepts to ensure results could be transferred to larger models with similar concepts. This setup provides the best possible selection of language models and ensures transferability to other models despite computational resource constraints.

This study focuses explicitly on time series imputation. Other time series tasks are not considered and should be examined in future work with similar detail. Hence, observations differ from previous studies like GPT4TS [26], which focused on developing a foundation model for various time series tasks. The results of this study suggest further investigation into other aspects. A key aspect is reproducing these findings using larger language models, such as Llama2 with 70 billion parameters, to confirm observations. This should include a complete fine-tuning of the model, as suggested by the analysis of optimizing individual model components. Additionally, considering the continuous development of new LLMs that achieve state-of-the-art results, future research should include these models (e.g., Google Gemma, Llama3 or Phi-3). Furthermore, other datasets should be explored. Although the datasets used in this study are based on previous research, they do not capture the full range of time series datasets. Particularly, datasets from different domains could provide new insights and should be included in future analysis.

Additionally, another aspect is to apply different sampling methods for simulating missing values. It would be of interest to observe the behavior of models when entire sequences within the time series are systematically removed and the process of removing observations is controlled.

6 Conclusion

Imputing incomplete time series is challenging but crucial for enabling valid downstream analysis. Due to its importance, several advanced methods based on the Transformer architecture have been developed, delivering state-of-the-art results. The success of LLMs in NLP motivated their application in the time series domain. This study successfully applied LLMs for time series imputation, showing that these models can serve as central components in the imputation process, with performance comparable to state-of-the-art models.

The experiments underscore that while model size and pre-training approaches are important, they are not the sole determinants of successful time series imputation. Large language models like Llama2 and Phi-2 consistently deliver strong performance across most scenarios. However, smaller models such as BERT and T5, when trained using the denoising autoencoding approach, demonstrate that specialized models with fewer parameters can achieve comparable, if not superior, performance to larger models. This suggests that the choice of model should be guided not solely by size but by the suitability of the model architecture and training approach for the specific task of imputation.

In practical terms, this means that researchers and practitioners can achieve high-quality imputations without relying exclusively on large models, which are often resource-intensive. Instead, specialized models optimized for the task can offer an efficient and effective alternative.

Furthermore, the fine-tuning experiments highlight that PEFT methods, particularly LoRA, are effective in adapting LLMs for time series imputation. This finding is consistent with broader NLP research, but specifically for time series imputation, it suggests that targeting the attention layer and feed-forward network during fine-tuning is critical for achieving optimal results. This insight is valuable for both academic research and practical implementations, as it provides a clear strategy for enhancing the performance of LLMs in this context.

Future work should address the limitations of this study, including exploring the findings in the context of other time series tasks, such as forecasting. The resource demands of the largest models, like Llama2 with 70 billion parameters, hinder the examination of even more complex models. Access to extensive resources would allow investigating the suitability of even larger models for time series imputation.

In summary, this study makes a significant contribution to time series imputation using LLMs. It demonstrates that language models are generally suitable for this task and provide a solid foundation. Future research may develop foundation models specifically trained for all time series tasks, potentially achieving similar success as language models in NLP tasks.

Funding Open Access funding enabled and organized by Projekt DEAL.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Ahn H, Sun K, Kim PK (2022) Comparison of missing data imputation methods in time series forecasting. Comput Mater Continua 70(1):767–779. https://doi.org/10.32604/cmc.2022.019369
2. Cao D, Jia F, Arik SO et al (2024) Tempo: prompt-based generative pre-trained transformer for time series forecasting. In: The twelfth international conference on learning representations
3. Chang C, Peng WC, Chen TF (2023) Llm4ts: aligning pre-trained llms as data-efficient time-series forecasters
4. Dettmers T, Pagnoni A, Holtzman A et al (2023) Qlora: efficient finetuning of quantized llms
5. Devlin J, Chang MW, Lee K et al (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In: North American chapter of the association for computational linguistics. Association for Computational Linguistics, pp 4171–4186
6. Du W, Côté D, Liu Y (2023) SAITS: Self-attention-based imputation for time series. Expert Syst Appl 219:119619. https://doi.org/10.1016/j.eswa.2023.119619
7. Gruver N, Finzi M, Qiu S et al (2023) Large language models are zero-shot time series forecasters. In: Thirty-seventh conference on neural information processing systems
8. Hu EJ, Shen Y, Wallis P et al (2022) Lora: Low-rank adaptation of large language models. In: International conference on learning representations
9. Hu Z, Wang L, Lan Y et al (2023) Llm-adapters: an adapter family for parameter-efficient fine-tuning of large language models. In: Proceedings of the 2023 conference on empirical methods in natural language processing. Association for Computational Linguistics, pp 5254–5276 https://doi.org/10.18653/v1/2023.emnlp-main.319
10. Javaheripi M, Bubeck S (2023) Phi-2: The surprising power of small language models. https://www.microsoft.com/en-us/research/blog/phi-2-the-surprising-power-of-small-language-models/. Accessed 19 Feb 2024
11. Jin M, Koh HY, Wen Q et al (2023) A survey on graph neural networks for time series: forecasting, classification, imputation, and anomaly detection https://doi.org/10.48550/ARXIV.2307.03759
12. Kazijevs M, Samad MD (2023) Deep imputation of missing values in time series health data: a review with benchmarking. J Biomed Inform 144:104440. https://doi.org/10.1016/j.jbi.2023.104440
13. Lewis M, Liu Y, Goyal N et al (2020) Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In: Proceedings of the 58th annual meeting of the association for computational linguistics. Association for Computational Linguistics, pp 7871–7880

14. Liu H, Tam D, Mohammed M et al (2022) Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. Adv Neural Inf Process Syst: 1950–1965
15. Osman MS, Abu-Mahfouz AM, Page PR (2018) A survey on data imputation techniques: water distribution system as a use case. IEEE Access 6:63279–63291. https://doi.org/10.1109/access.2018.2877269
16. Pu G, Jain A, Yin J et al (2023) Empirical analysis of the strengths and weaknesses of peft techniques for llms. In: ICLR 2023 workshop on mathematical and empirical understanding of foundation models
17. Radford A, Wu J, Child R et al (2019) Language models are unsupervised multitask learners
18. Raffel C, Shazeer N, Roberts A et al (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(1)
19. Touvron H, Martin L, Stone K et al (2023) Llama 2: open foundation and fine-tuned chat models
20. Vaswani A, Shazeer N, Parmar N et al (2017) Attention is all you need. In: Proceedings of the 31st international conference on neural information processing systems, NIPS'17, vol 30. Curran Associates Inc, pp 6000–6010
21. Wu H, Hu T, Liu Y et al (2023) Timesnet: temporal 2d-variation modeling for general time series analysis. In: International conference on learning representations
22. Zhang Q, Chen M, Bukharin A et al (2023) Adaptive budget allocation for parameter-efficient fine-tuning. In: The eleventh international conference on learning representations
23. Zhao B, Tu H, Wei C et al (2023) Tuning layernorm in attention: towards efficient multi-modal llm finetuning. In: The twelfth international conference on learning representations. arxiv
24. Zhao WX, Zhou KLJ (2023) A survey of large language models https://doi.org/10.48550/ARXIV.2303.18223
25. Zhou H, Zhang S, Peng J et al (2021) Informer: beyond efficient transformer for long sequence time-series forecasting. In: The thirty-fifth AAAI conference on artificial intelligence, AAAI 2021, virtual conference, vol 35. AAAI Press, pp 11106–11115 https://doi.org/10.48550/ARXIV.2012.07436
26. Zhou T, Niu P, Wang X et al (2023) One fits all: power general time series analysis by pretrained lm. In: Advances in neural information processing systems, vol 36. Curran Associates, Inc, pp 43322–43355

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.